```sql
INSERT INTO topic_statistics (views, id) VALUES (1, 2)

INSERT INTO topic_statistics (views, id) VALUES (1, 3)
```

Unlike the joined table inheritance, each persisted subclass entity requires a single insert statement.

When fetching all `Topic` entities associated with a specific `Board`:

```java
List<Topic> topics = entityManager.createQuery(
    "select t from Topic t where t.board.id = :boardId", Topic.class)
.setParameter("boardId", 1L)
.getResultList();
```

Hibernate uses `UNION ALL` to fetch rows from the base class and every subclass table in this particular inheritance tree.

```sql
SELECT
    t.id AS id1_3_,
    t.board_id AS board_id5_3_,
    t.createdOn AS createdO2_3_,
    t.owner AS owner3_3_,
    t.title AS title4_3_,
    t.content AS content1_2_,
    t.validUntil AS validUnt1_0_,
    t.clazz_ AS clazz_
FROM (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id, content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content, validUntil, 2 AS clazz_
    FROM announcement ) t
WHERE  t.board_id = 1
```

When loading a `TopicStatistics` while also fetching its associated `Topic` relation, Hibernate must use `UNION ALL` for the inheritance tables to construct the desired result set:

```
TopicStatistics statistics = entityManager.createQuery(
    "select s from TopicStatistics s join fetch s.topic t where t.id = :topicId"
    , TopicStatistics.class)
.setParameter("topicId", topicId)
.getSingleResult();
```

```
SELECT
    ts.id AS id1_4_0_, t.id AS id1_3_1_, ts.views AS views2_4_0_,
    t.board_id AS board_id5_3_1_, t.createdOn AS createdO2_3_1_,
    t.owner AS owner3_3_1_, t.title AS title4_3_1_, t.content AS content1_2_1_,
    t.validUntil AS validUnt1_0_1_, t.clazz_ AS clazz_1_
FROM topic_statistics ts
INNER JOIN (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        validUntil, 2 AS clazz_
    FROM announcement
) t ON ts.id = t.id
WHERE t.id = 2
```

The identity generator is not allowed with this strategy because rows belonging to different subclasses would share the same identifier, therefore conflicting in polymorphic `@ManyToOne` or `@OneToOne` associations.

Considering that the `Board` entity defines a `@OneToMany` `Topic` association:

```
@OneToMany(mappedBy = "board")
private List<Topic> topics = new ArrayList<>();
```

Fetching the collection lazily generates a separate select statement, identical to the previous `Topic` entity query.

When fetching the `topics` collection eagerly:

```
Board board = entityManager.createQuery(
    "select b from Board b join fetch b.topics where b.id = :id", Board.class)
.setParameter("id", id)
.getSingleResult();
```

Hibernate requires a join with the result of unifying all three topic-related tables.

```
SELECT
    b.id AS id1_1_0_, t1.id AS id1_3_1_, b.name AS name2_1_0_,
    t1.board_id AS board_id5_3_1_, t1.createdOn AS createdO2_3_1_,
    t1.owner AS owner3_3_1_, t1.title AS title4_3_1_,
    t1.content AS content1_2_1_, t1.validUntil AS validUnt1_0_1_,
    t1.clazz_ AS clazz_1_, t1.board_id AS board_id5_3_0__, t1.id AS id1_3_0__
FROM board b
INNER JOIN (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        validUntil, 2 AS clazz_
    FROM announcement
) t1 ON b.id = t1.board_id
WHERE b.id = 1
```

Performance considerations

While write operations are faster than for the joined table strategy, the read operations are only efficient when querying against the actual subclass entities. Polymorphic queries can have a considerable performance impact because Hibernate must select all subclass tables and use `UNION ALL` to build the whole inheritance tree result set. As a rule of thumb, the more subclass tables, the less efficient the polymorphic queries get.

11.4 Mapped superclass

If the `Topic` class is not required to be a stand-alone entity, it will be more practical to leave inheritance out of the database. This way, the `Topic` can be made `abstract` and marked with the `@MappedSuperclass` annotation so that JPA can acknowledge the inheritance model on the entity-side only.

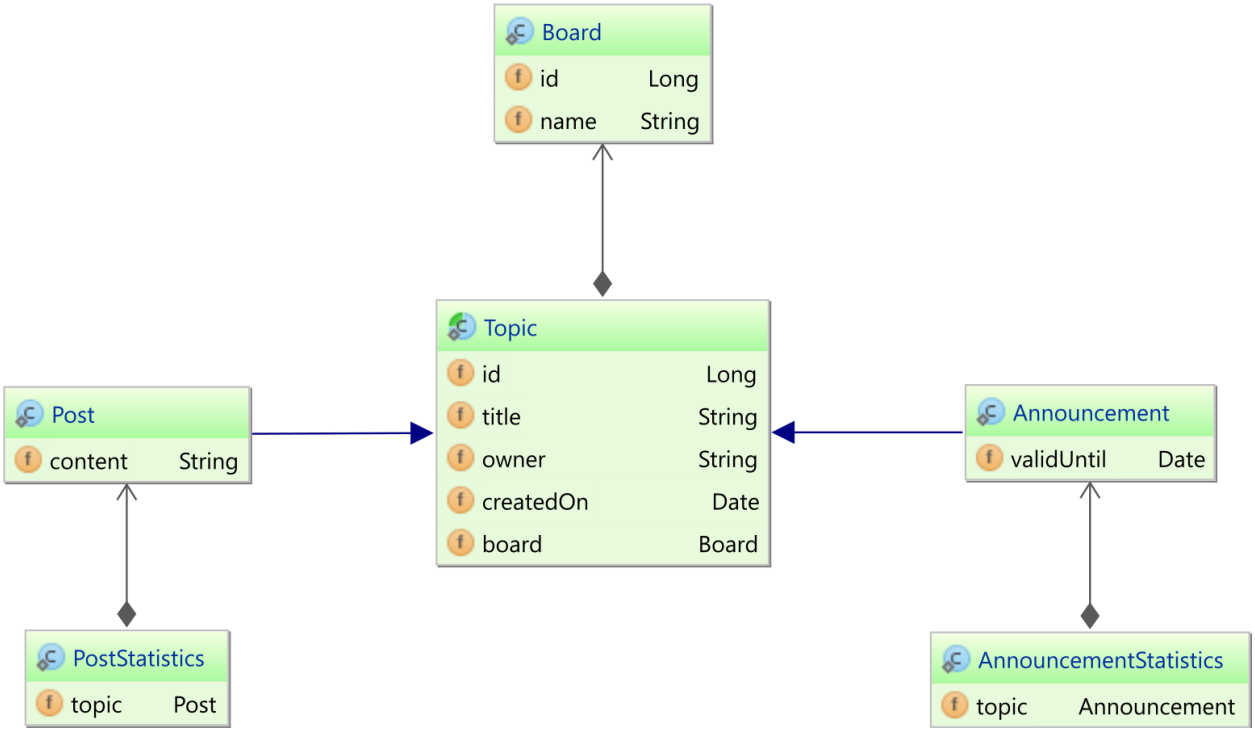

Figure 11.5: Mapped superclass class diagram

Having a single `TopicStatistics` entity with a `@OneToOne Topic` association is no longer possible because the `Topic` class is not an entity. This way, each `Topic` subclass must define its own statistics associations (e.g. `PostStatistics`, `AnnouncementStatistics`).

The `PostStatistics` and the `AnnouncementStatistics` entities looks as follows:

```
@Entity @Table(name = "post_statistics")
public class PostStatistics
    extends TopicStatistics<Post> {

    @OneToOne
    @JoinColumn(name = "id")
    @MapsId
    private Post topic;

    private PostStatistics() {}

    public PostStatistics(Post topic) {
        this.topic = topic;
    }

    @Override
    public Post getTopic() {
        return topic;
    }
}

@Entity @Table(name = "announcement_statistics")
public class AnnouncementStatistics
    extends TopicStatistics<Announcement> {

    @OneToOne
    @JoinColumn(name = "id")
    @MapsId
    private Announcement topic;

    private AnnouncementStatistics() {}

    public AnnouncementStatistics(Announcement topic) {
        this.topic = topic;
    }

    @Override
    public Announcement getTopic() {
        return topic;
    }
}
```